EXATHON INDIA 2026

CloseDesk

The governed Exasol agent for UPI reconciliation.

Name

Sharma S S

Team

Late Orbit

Topic

Money, Risk & Regulation

Company

Wekan Enterprise Solutions

github.com/sharma-10-personal/closedesk

MIT licence · synthetic data only

THE PROBLEM

Every day, three lists disagree — and an analyst untangles them by hand

NPCI settlement

What the network says happened to each UPI payment.

Bank ledger

What core banking actually debited or credited.

Merchant records

What the merchant and gateway think they were paid.

> "I start the day with thousands of mismatches across spreadsheets. By the time I've traced them, the refund deadline for a failed-but-debited customer may already have passed."

THE GOAL

Start the day with a short list of real exceptions — each with a cause, ₹ at risk and a proposed fix

- Most rows match automatically, inside the database, in seconds.
- Leftovers are classified into named break types, not "unmatched".
- Breaks are grouped by root cause, so one fix can clear thousands of rows.
- The queue is ranked by rupees at risk and deadline, not by row order.
- An agent proposes the fix. A human approves or rejects. Everything is audited.

HOW IT WORKS

One pipeline, all of it in Exasol SQL

01
Load
Generate a synthetic day or upload three CSVs. Bulk import with pyexasol.

›

02
Match
Exact, then amount/time tolerance, then many-to-one batch settlements.

›

03
Classify
Missing, duplicate, amount mismatch, failed-but-debited, reversal not posted.

›

04
Group
Root causes by gateway × break type, ranked by ₹ at risk and deadline.

›

05
Propose
Agent maps each break to an allowed action. Policy gate decides allow, escalate or block.

›

06
Decide
Human approves or rejects. Decision and audit row written to Exasol.

Nine SQL stages in `sql/`. No Python loops over transactions, no UDFs, no LLM key required.

EFFECTIVE USE OF EXASOL PERSONAL

“The sovereign analytics database, built for the agentic era.” CloseDesk uses all three parts.

Sovereign by default

NPCI, ledger and merchant files, matches, proposals, decisions and the audit log all live in one local `CLOSEDESK` schema. No cloud account, no API key, nothing sent out.

In-memory SQL

Nine plain SQL stages do the whole match, classification, grouping and ranking. No Python loop over transactions. 6.65s for ~3M rows across three sources on a 2-vCPU local VM.

Agents on your own data

The agent reads `breaks` from Exasol and writes `proposals` back. The gate decision, the human approval and `audit_log` are Exasol rows too.

- Python only validates files, bulk-loads them with `pyexasol`, and serves the UI. Every screen queries live Exasol state.
- Not used yet: UDFs and S3/Iceberg connectors. Next: run the same SQL on a multi-node BYOC cluster at 10M+ rows.

Quote: exasol.com/personal

MEASURED ON A 1,000,000-TRANSACTION DAY (~1M ROWS PER SOURCE)

Fast enough to run while the answer still matters

99.1%
AUTO-MATCHED

6.65s
9 SQL STAGES, END TO END

100%
PRECISION VS ANSWER KEY

99.79%
RECALL VS ANSWER KEY

STAGE	ROWS	TIME
Exact match	977,846	3.92s
Tolerance match	70	0.57s
Many-to-one batch	2,667	0.68s
Break detection (4 stages)	31,417	1.35s

Exasol Personal 2026.2.0 · local VM, 2 vCPU · MacBook

FROM ROWS TO CAUSES

31,417 breaks. 31 root causes.

989,666	980,583	31,417	31
NPCI ROWS	MATCHED	BREAKS	ROOT CAUSES

Each cause is a gateway × break type group, sized by rupees at risk. The analyst fixes the cause once — a gateway clock, a duplicate posting job — instead of chasing thousands of rows.

GOVERNED AGENT

The agent proposes. A human decides. Exasol remembers.

Allow · ≤ ₹50,000

Offered for one-click approval by the analyst.

Escalate · up to ₹5,00,000

Flagged for a supervisor before anyone acts.

Block · above that

Approve is disabled in the UI and refused in code. Manual process only.

- The agent can only choose from an allow-list of five actions in `config/policy.yaml`.
- Every proposal, approval and rejection is written to `audit_log` with before and after state.
- Explanations come from templates and SQL labels, so it runs with no LLM and no API key.

THE PRODUCT

Five screens, all reading live Exasol state

Pipeline

Generate a synthetic day or upload your three CSVs, validated before load. Watch each SQL stage run.

Investigate

One NPCI reference across all three sources, its match or break, and the proposed fix.

Inject

Plant a new break into the live data and watch CloseDesk detect and propose a fix in seconds.

Exceptions

The proposal queue ranked by ₹ at risk, filtered by type, gate and status. Approve or reject.

Analytics

Funnel, break types, root causes, accuracy against the answer key, and the full audit trail.

Try it

```
make setup
make full
make demo
```

WHAT IT DOES NOT DO YET

Honest limits and next steps

Limits today

83 timing-skew cases that cross midnight are missed (recall 99.79%).

Approval records the decision; it does not post to a core-banking system.

The agent runs as the database owner, not a read-only user.

All data is synthetic. The compensation rate is a placeholder.

Next

Cross-midnight tolerance window.

Read-only database role for the agent.

Undo for decisions, and posting via a maker-checker integration.

Generate and match 10M+ rows entirely in Exasol SQL.

CLOSEDESK

Close the day before lunch, not after midnight.

Open source under MIT. Exasol Personal, Python, Streamlit. All transactions synthetic.

github.com/sharma-10-personal/closedesk